

Brandy Gonzales

Supervised Learning is practice with an answer key. You show the computer the data and the correct label, then it guesses new data. In this lab that meant wine numbers plus the wine type, then a guess on a new wine. Unsupervised learning has no answer key. You only hand over the numbers, and the computer groups things that look alike, like customers who buy the same stuff. Reinforcement learning is trial and error. The computer tries something, gets a reward or a penalty, and tries again, the way a robot learns chess.

The idea that clicked was the answer key. Supervised learning has the label during training. Unsupervised learning does not. The split mattered too. The model learned on 142 wines and was graded on 36 it had never seen, so 88.9% only counts on those new wines.

Logistic Regression was the best of the two original models. It got 88.9% right, and the Decision Tree got 83.3%. Logistic Regression looked at all four wine numbers at once. The tree was only allowed three yes-or-no questions, so it lumped some wines together and missed more of them.

I then tried other sets of three features. Proline, total phenols, and flavonoids scored 91.7%. Proline, total phenols and od280 also scored 91.7%. Both beat 88.9%. Sets that included malic acid scored lower, down to 66.7%, and the best ones reached 91.7%.

A healthcare use of the same idea would be illness diagnosis. The model would need symptoms, lab numbers, and the illness a doctor already confirmed. The symptoms and labs are features. The confirmed illness is the answer key. A doctor would still make the final call.

Questions I still have: would all 13 features score higher than 91.7%, or would extra numbers confuse the model? What happens if the tree is allowed more than three questions? Could a loop test every set of three features, all 286 of them, instead of guessing?

Derrick K. Gibbs-McGlaston

My Understanding of Machine Learning Types

The biggest difference I noticed between the three learning types is where the learning signal comes from. In supervised learning, the training data includes the correct answer, so the model learns how input features relate to a known target. The Wine Classification project is a clear example because the wine class is already known for the training examples. Unsupervised does not provide those answers. Instead, the model looks for patterns or groups in the data. Reinforcement learning is different because an agent learns by taking actions and receiving feedback from an environment. Thinking about these three approaches in terms of the information available during learning made the differences much easier for me to understand.

What I Learned from the ML Workflow

The workflow initially looked like a long list of separate tasks, but completing the notebook helped me see how each stage depends on the previous one. Data preparation determines what information the model receives. Splitting the data creates a separate test set so performance can be checked on examples the model did not train on. Training is only the middle of the process; evaluation is what tells us whether the learned patterns are useful. The confusion matrix also showed me that accuracy by itself does not explain every type of mistake a model makes.

Analysis of the Wine Classification Project

The original Logistic Regression model achieved 88.9% accuracy on the 36-example test set, while the Decision Tree achieved 83.3%. The classification report showed that the models did not perform equally across all three wine classes. In the hands-on experiment, changing the three selected features to alcohol, hue, and proline increased the Logistic Regression test accuracy to 97.2% on the same test split. That result was useful because it demonstrated how feature selection can affect model performance. At the same time, I learned not to assume that one test split proves a model will always perform that well. I would want to use cross-validation and additional testing before making that conclusion.

Challenges and Problem-Solving

One important challenge was understanding why the test data must remain separate from the training data. I now understand that if a model is evaluated on data it has already seen, the

performance measurement can give a misleading picture of how the model will perform on new data. Another useful lesson was seeing that changing the input features changed the result. Instead of treating the model as a black box, I can now think about the entire pipeline: what problem is being solved, what data is available, which features are useful, how the data is divided, and how the results are measured.

Connection to Future Work

The workflow connects directly to the type of work I want to do with AI and programming. A real project would not stop after training a model. It would require defining the problem, collecting and checking data, selecting features, evaluating performance, documenting limitations, deploying the model, and monitoring it over time. I also want to explore feature scaling, cross-validation, hyperparameter tuning, and ways to detect when real-world data changes enough that a model needs to be retrained.

Questions for Further Exploration

The questions I want to explore next are: How does cross-validation change confidence in a model's performance? How do I decide which features should be kept or removed? When is Logistic Regression preferable to a tree-based model? How can a model be monitored after deployment? Finally, how should performance be measured when the cost of one type of prediction error is bigger than another?

Marlon Roberts

Reflection and Analysis

My Understanding of Machine Learning Types

Supervised Learning: Supervised learning is practice with an answer key. You show the computer the data, then it guesses new data. Like showing it wine numbers plus the wine type, then asking it to guess a new wine.

Unsupervised Learning: Unsupervised learning has no answer key. You only hand over the numbers and the computer groups things that look alike, like sorting customers who buy the same stuff.

Reinforcement Learning: Reinforcement learning is trial and error. The computer tries something, gets a reward or a penalty, and tries again. Like how a robot learns chess.

My Analysis of the Wine Classification Project

Logistic Regression was the best. It got **88.9% right**. The Decision Tree got **83.3%**.

Logistic Regression looked at all four wine numbers at once and drew one smooth line between the types. The Decision Tree was only allowed three yes-or-no questions, so it had to lump some wines together and missed more of them.

I tried other sets of three features. Proline, total phenols, and flavonoids got **91.7%**, and proline, total phenols, and OD280 also scored **91.7%**. Both beat the original model at **88.9%**. The sets that included malic acid scored lower, down to **66.7%**. I would keep proline and total phenols because those two carried the improvement, and the third feature could be flavonoids or OD280.

Real-World Application Ideas

Industry of Interest: Healthcare

ML Problem: Illness diagnosis

Type of ML: Supervised Model

Data Needed: Symptoms, lab numbers, and the illness the doctor already confirmed.

Pablo Lara

Reflection and Analysis

My Understanding of Machine Learning Types

Supervised Learning: Supervised learning is when the model is given data along with the correct answers. It learns patterns from those examples and then uses what it learned to make predictions on new data it has not seen before.

Unsupervised Learning: Unsupervised learning is when the model is given data without the correct answers. The model looks through the data to find patterns, similarities, or groups on its own.

Reinforcement Learning: Reinforcement learning is when an AI learns through trial and error. It receives rewards or penalties based on its actions and uses that feedback to learn which actions work better.

My Analysis of the Wine Classification Project

Best performing model: Logistic Regression

Why do you think this model performed better?: I think Logistic Regression was able to find useful relationships between the features we selected and the different wine classes. When we tested the model using data it had not trained on, it was able to correctly classify more of the wine samples.

What would you try next to improve performance?: I would try different combinations of features and compare the results. One thing I learned from this lab is that the features we choose can affect how well the model performs, so I would want to experiment with them and see which combinations give the model the most useful information.

Real-World Application Ideas

Industry of Interest: Cybersecurity

ML Problem: I would be interested in using machine learning to help identify potentially malicious network activity. A model could examine network activity and help determine whether the activity appears normal or potentially malicious.

Type of ML: Supervised Learning

Data Needed: I would need examples of normal and malicious network activity that have already been labeled. The data could include information such as network connections, ports, traffic activity, and other characteristics that could help distinguish normal activity from malicious activity.

Key Learnings

Most important concept learned: The most important concept I learned was the difference between exploring data and actually training a machine-learning model. At first, I thought that exploring and graphing the data was already machine learning. I now understand that EDA helps me understand the data first. When we use the training data and the `.fit()` command, that is when the model actually starts learning patterns from the examples.

Most challenging part: The most challenging part was understanding how all of the different steps connect together. At first, I was not completely sure when the computer was actually learning. Going through the workflow helped me understand that we first explore and prepare the data, select features, split the data, train the model, and then test it using data it has not seen before.

Questions for further exploration: I would like to learn more about how we decide which features are the most useful for training a model. I would also like to understand how we can tell if a model is actually learning useful patterns instead of just getting good results from one particular set of data.